PYTHON BOOLEANS

Clear classroom notes with examples and practice

Learning objective: Understand what Boolean values are, how Python produces them, how to test values with `bool()`, and how to use conditions in programs.

1. What is a Boolean?

A **Boolean** is a data type that represents one of only two possible values: `True` or `False`. In simple terms, a Boolean answers a yes-or-no question.

For example: Is 10 greater than 5? Yes, so the answer is `True`. Is 10 equal to 3? No, so the answer is `False`.

```
print(10 > 5)    # True
print(10 == 3)   # False
print(7 < 9)     # True
```

Important: In Python, write Boolean values with a capital first letter: `True` and `False`. The lowercase forms `true` and `false` are not valid Boolean keywords.

2. Comparisons produce Boolean results

Comparison operators compare two values. The result of every comparison is either `True` or `False`.

Operator	Meaning	Example
==	Equal to	5 == 5 → True
!=	Not equal to	5 != 3 → True
>	Greater than	8 > 4 → True
<	Less than	2 < 1 → False
>=	Greater than or equal to	6 >= 6 → True
<=	Less than or equal to	3 <= 2 → False

Example: compare two variables

```
age = 20
minimum_age = 18

print(age >= minimum_age)  # True
```

The expression checks whether `age` is at least 18. Because 20 is greater than 18, Python returns `True`.

3. Using Booleans with if and else

Programs often need to make decisions. An `if` statement runs a block of code when its condition is `True`. An `else` block runs when the condition is `False`.

```
temperature = 32

if temperature > 30:
    print("It is a hot day")
else:
    print("The temperature is not above 30")
```

Python evaluates `temperature > 30`. Since the result is `True`, the first message is printed.

4. The bool() function

The built-in `bool()` function converts a value into its Boolean equivalent. It returns either `True` or `False`.

```
print(bool("Python"))  # True
print(bool(25))       # True
print(bool(0))        # False
```

You can also store values in variables and test them:

```
city = "Hyderabad"
items_in_cart = 0

print(bool(city))           # True
print(bool(items_in_cart))  # False
```

5. Truthy values: values that become True

Most non-empty values are considered **truthy**. This means `bool(value)` returns `True`.

```
bool("Hello")                # True: non-empty string
bool(100)                   # True: non-zero number
bool(["red", "blue"])       # True: non-empty list
bool((10, 20))              # True: non-empty tuple
bool({"name": "Asha"})      # True: non-empty dictionary
bool({1, 2, 3})             # True: non-empty set
```

A value does not need to be the Boolean value `True` to be truthy. For example, the string `'Hello'` is a string, but Python treats it as true when checking its truth value.

6. Falsy values: values that become False

The most common falsy values are zero, empty collections, an empty string, `None`, and `False`.

```
bool(False)   # False
bool(None)    # False
bool(0)       # False
bool(0.0)     # False
bool("")      # False: empty string
bool([])      # False: empty list
bool(())      # False: empty tuple
bool({})      # False: empty dictionary
bool(set())   # False: empty set
```

Easy rule to remember: A non-empty string or collection is usually truthy. An empty string or collection is falsy. The number zero is falsy; non-zero numbers are truthy.

7. Boolean values returned by functions

A function can return a Boolean value. This is useful when a function checks something and reports whether the check passed.

```
def is_adult(age):
    return age >= 18

result = is_adult(21)
print(result)  # True

if is_adult(16):
    print("Access allowed")
else:
    print("Access denied")
```

Here, `is_adult()` returns `True` when the age is 18 or above; otherwise, it returns `False`.

8. Checking a data type with isinstance()

The built-in `isinstance()` function checks whether a value belongs to a specified data type. It returns a Boolean result.

```
score = 95
name = "Ravi"

print(isinstance(score, int))  # True
print(isinstance(name, int))   # False
print(isinstance(name, str))   # True
```

9. Custom objects and Boolean behaviour

Python can also determine an object's truth value using special methods. If a class defines `__bool__()`, Python uses its returned Boolean value. If `__bool__()` is not defined, Python can use `__len__()`: a length of zero is falsy, while a non-zero length is truthy.

```
class Basket:
    def __len__(self):
        return 0

my_basket = Basket()
print(bool(my_basket))  # False
```

This example returns `False` because the object's length is zero.

10. Common mistakes to avoid

• Use == to compare values; use = to assign a value.
• Write `True` and `False` with capital letters.
• Do not assume that only Boolean values can be used in an `if` condition. Strings, numbers, lists, and other objects can also be tested.
• Remember that an empty list `[]` is falsy, but a list containing an item is truthy.

11. Quick practice

Predict the output before running each example.

```
# Question 1
print(bool(""))

# Question 2
print(bool("0"))

# Question 3
print(12 != 12)

# Question 4
numbers = []
if numbers:
    print("List has items")
else:
    print("List is empty")

# Question 5
value = 14
print(isinstance(value, int))
```

Answers

1. `False` — empty string.
2. `True` — the string contains the character 0; it is not an empty string.
3. `False` — 12 is equal to 12.
4. `List is empty` — an empty list is falsy.
5. `True` — 14 is an integer.

Key takeaways

• Boolean is a Python data type with two values: `True` and `False`.
• Comparisons produce Boolean results.
• `if` and `else` use conditions to control program flow.
• `bool()` checks the truth value of any object.
• Zero, `None`, `False`, and empty collections are common falsy values.
• Functions such as `isinstance()` can return Boolean values.